
Bearing Digital Twin

A Project Report submitted in partial fulfilment of
ME F434 – Digital Twins in Mechanical Engineering

Group Number: 5

Members: Ritesh Nallamalli (2023ABPS0804P)
Pranav Girish Morab (2023ABPS0797P)
Goutham Mahesh (2023A7PS0674P)

Institution: BITS Pilani, Pilani Campus

Date: April 2026

Contents

1	Introduction	2
1.1	Objectives	2
2	Dataset Description	2
3	Feature Engineering	3
3.1	Time-Domain Feature Extraction	3
3.1.1	Root Mean Square (RMS)	3
3.1.2	Kurtosis	3
3.1.3	Crest Factor	3
3.2	Frequency-Domain Analysis (FFT)	4
3.3	Rolling Smoothing	4
4	PCA-Based Health Indicator Model	5
4.1	Why PCA for Health Indicator Synthesis?	5
4.2	Two-Phase Fitting Procedure	6
4.3	Online Transform (<code>transform_daily</code>)	6
4.4	Feature Selection via Monotonicity	7
5	Frequency Spectrum Analysis	8
5.1	Bearing Fault Frequencies	8
5.2	Spectral Evolution Across 50 Days	9
6	System Architecture & Communication	10
6.1	Overall Architecture	10
6.2	Backend: Flask REST API on Raspberry Pi (<code>server3.py</code>)	10
6.2.1	Threading model and shared state	11
6.2.2	Simulation timing and data flow	11
6.2.3	API endpoints	11
6.3	Frontend: Browser Dashboard (<code>index.html</code>)	12
6.4	Fusion 360 3-D Visualisation (<code>Color.py</code>)	12
6.4.1	Component discovery	13
6.4.2	Polling and cache busting	13
6.4.3	Health-to-colour mapping	13
6.4.4	Day-change guard and completion	13
6.4.5	Network error handling	14
7	Results and Discussion	14
7.1	Health Indicator Performance	14

7.2	Feature Analysis	14
7.3	Spectral Observations	14
7.4	Digital Twin Communication Latency	14
8	Key Code Explanation	15
8.1	Feature Extraction (<code>extract_time_features</code>)	15
8.2	PCA Health Indicator Transform (<code>transform_daily</code>)	15
8.3	Fusion 360 Colour Mapping (<code>Color.py</code>)	15
9	Conclusion	16

1 Introduction

Rolling-element bearings are among the most critical and failure-prone components in wind turbines. An undetected inner-race or outer-race fault can escalate from microscopic surface fatigue to catastrophic mechanical failure within days, causing costly unplanned downtime and significant repair expenditure. Traditional time-based maintenance schedules are wasteful; condition-based maintenance requires robust real-time health indicators derived directly from the machine's operating signals.

A **Digital Twin** (DT) offers a compelling solution: a live, software replica of the physical asset that continuously ingests sensor data, applies signal-processing and machine-learning algorithms, and exposes actionable health metrics to operators through an interactive interface. This project implements a complete end-to-end Digital Twin for a wind turbine high-speed shaft bearing, using the publicly available 50-day run-to-failure vibration dataset [1]. The system spans data ingestion, feature engineering, PCA-based health indicator synthesis, a REST API backend, a browser dashboard, and a live-coloured Autodesk Fusion 360 3-D model.

1.1 Objectives

- Extract time-domain and frequency-domain features from raw vibration signals sampled at 97.6 kHz.
- Construct a monotonicity-guided, PCA-fused Health Indicator (HI) that degrades monotonically from 1.0 (healthy) to 0.0 (failed).
- Deliver HI updates and FFT spectra in real time via a Flask REST API running on a Raspberry Pi.
- Visualise degradation trends, frequency spectra, and time-domain waveforms in an interactive browser dashboard.
- Drive a live colour-mapped 3-D bearing model in Autodesk Fusion 360 to provide an intuitive engineering view of fault progression.

2 Dataset Description

The dataset was sourced from a 2 MW wind turbine high-speed shaft bearing (model 624ZZ class) driven by a 20-tooth pinion gear [1]. One vibration recording of approximately 6 seconds was acquired per calendar day for **50 consecutive days**, during which an inner-race fault nucleated and propagated to complete failure.

Table 1. Dataset summary

Property	Value	Notes
Number of files	50	One <code>.mat</code> file per day
Sampling frequency	97,656 Hz	High-speed shaft vibration
Signal length	$\approx 585,000$ samples	≈ 6 s per file
Fault type	Inner-race fatigue	Progresses to full failure
Tachometer PPR	2	Pulses per revolution
FFT analysis window	0–10,000 Hz	Region of interest for fault harmonics

3 Feature Engineering

3.1 Time-Domain Feature Extraction

Three scalar features are extracted from each daily raw vibration signal $x[n]$ of length N , after removing the DC offset:

3.1.1 Root Mean Square (RMS)

RMS measures the overall energy content of the signal and grows steadily as bearing surface damage increases friction and vibration amplitude [3].

$$\text{RMS} = \sqrt{\frac{1}{N} \sum_{n=1}^N x^2[n]} \quad (1)$$

3.1.2 Kurtosis

Kurtosis quantifies the “peakedness” of the signal amplitude distribution [3]. A healthy bearing exhibits Gaussian-like vibration with kurtosis near 3.0 (normalised, Fisher = False). The appearance of impulsive ring-downs caused by ball impacts on a spalled inner race drives kurtosis well above this baseline.

$$K = \frac{\frac{1}{N} \sum_{n=1}^N (x[n] - \bar{x})^4}{\left(\frac{1}{N} \sum_{n=1}^N (x[n] - \bar{x})^2 \right)^2} \quad (2)$$

3.1.3 Crest Factor

Crest Factor (CF) is the ratio of the signal’s peak absolute value to its RMS [4]. It is particularly sensitive to early-stage impulsive faults before the RMS has risen noticeably, making it a valuable early-warning indicator.

$$CF = \frac{\max(|x[n]|)}{\text{RMS} + \epsilon} \quad (3)$$

where $\epsilon = 10^{-12}$ prevents division by zero.

3.2 Frequency-Domain Analysis (FFT)

A Fast Fourier Transform (FFT) is computed on each daily signal after applying a **Hanning window** to reduce spectral leakage:

$$A(f) = \frac{2}{N} |\mathcal{F}\{x[n] \cdot w[n]\}| \quad (4)$$

where $w[n] = 0.5 \left(1 - \cos\left(\frac{2\pi n}{N-1}\right)\right)$ is the Hanning window. The spectrum is truncated to $f \leq 10,000$ Hz, where bearing fault harmonics of BPFI, BPFO, and BSF are most prominent. The FFT output is downsampled to ≈ 500 points for efficient network transmission.

3.3 Rolling Smoothing

A 5-day backward-looking rolling mean is applied to all three raw feature vectors before PCA input, suppressing day-to-day measurement noise while preserving the long-term degradation trend:

$$\tilde{\mathbf{f}}_i = \frac{1}{\min(i, 5)} \sum_{k=\max(1, i-4)}^i \mathbf{f}_k \quad (5)$$

Figure 1 shows the evolution of all three features across the 50-day life, with three distinct phases clearly visible: stable baseline (days 1–20), gradual rise (days 20–40), and accelerated degradation (days 40–50).

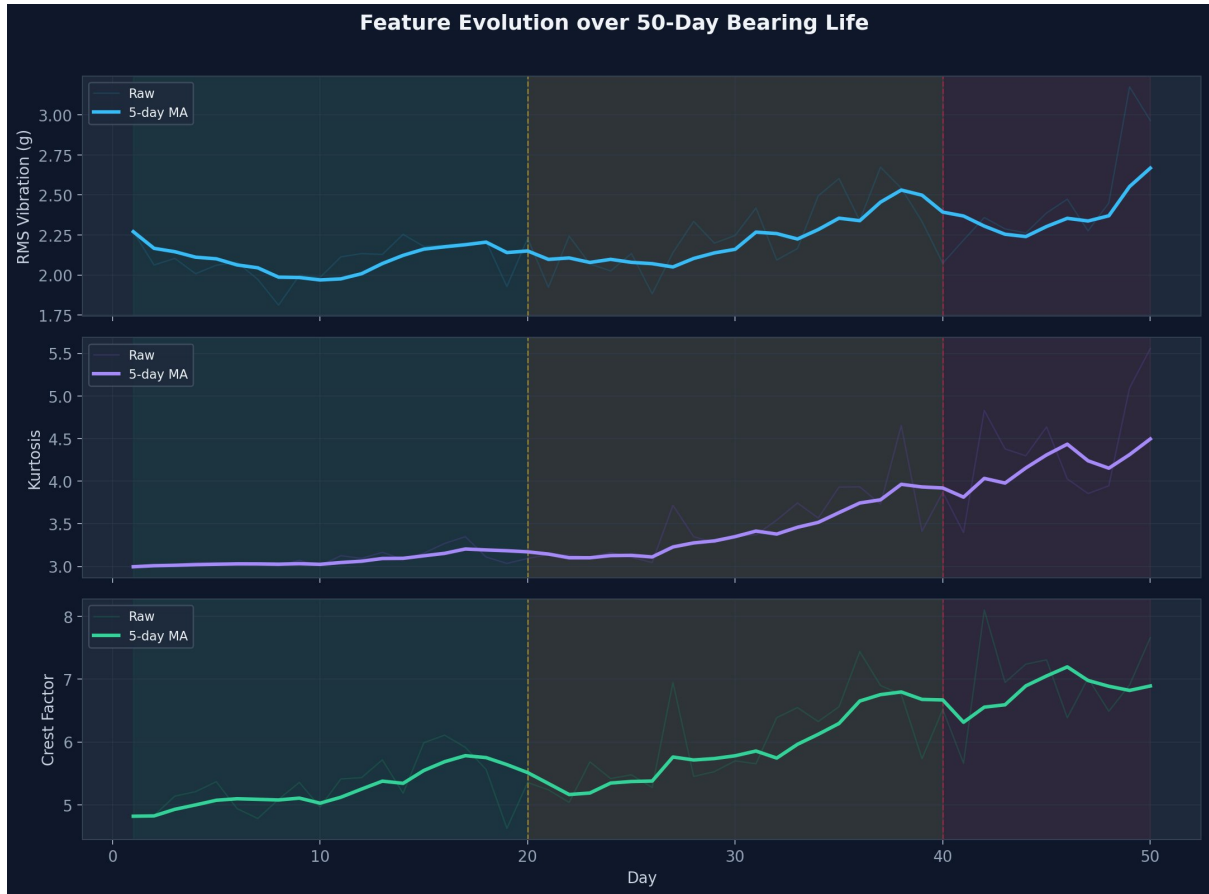


Figure 1. Evolution of RMS, Kurtosis, and Crest Factor over 50 days. Dashed yellow and red lines mark the fault onset (day 20) and rapid degradation (day 40) boundaries. Faint lines show raw daily values; bold lines show the 5-day moving average.

4 PCA-Based Health Indicator Model

4.1 Why PCA for Health Indicator Synthesis?

Principal Component Analysis (PCA) [2] is an unsupervised linear dimensionality-reduction technique. Given a matrix of observations $\mathbf{X} \in \mathbb{R}^{N \times p}$ (here $N = 20$ training days and $p = 2$ selected features), PCA finds an orthogonal basis – the *principal components* – that maximises the explained variance. The first principal component (PC1) is the direction of greatest variance in the feature space.

In the context of bearing health monitoring, a key insight is that during the *healthy training period* the feature cloud occupies a compact, low-variance region. As faults develop, feature values deviate from this baseline along consistent directions, causing the PC1 projection to grow. This makes the distance from the healthy-baseline PC1 value a natural scalar health indicator: small distance = healthy, large distance = degraded.

Using PCA rather than any single feature provides two advantages. First, it exploits the *correlation structure* between Kurtosis and Crest Factor: both rise together as fault

energy grows, and PC1 is precisely the direction that captures this joint rise. Second, it is inherently *unsupervised* – no labelled failure data are required to fit the model, only the first 20 healthy-state days.

4.2 Two-Phase Fitting Procedure

The model fitting (`fit()` method) operates in two sequential phases.

Phase 1 – PCA training. Features are extracted and smoothed for the first 20 days. The selected features (Kurtosis and Crest Factor, chosen by monotonicity scoring as described in Section 4.4) are *z-score normalised* using the training-set mean $\boldsymbol{\mu}_{\text{train}}$ and standard deviation $\boldsymbol{\sigma}_{\text{train}}$:

$$\mathbf{z}_i = \frac{\tilde{\mathbf{f}}_i^{(\text{sel})} - \boldsymbol{\mu}_{\text{train}}}{\boldsymbol{\sigma}_{\text{train}}} \quad (6)$$

PCA is then fitted on $\{\mathbf{z}_1, \dots, \mathbf{z}_{20}\}$. The first eigenvector $\mathbf{w}_1$ of the sample covariance matrix is stored. The healthy baseline PC1 score is recorded as:

$$s_{\text{baseline}} = \mathbf{w}_1^\top \mathbf{z}_1 \quad (7)$$

Phase 2 – Global scale discovery. All 50 days are processed using the already-fitted scaler and PCA to compute the raw deviation:

$$\delta_i = |s_i - s_{\text{baseline}}|, \quad s_i = \mathbf{w}_1^\top \mathbf{z}_i \quad (8)$$

The maximum deviation $\delta_{\text{max}} = \max_i \delta_i$ (occurring at or near Day 50) is stored. This pre-scan ensures the final HI is properly scaled: the most degraded observation maps to HI = 0 and the healthy baseline maps to HI = 1. Without this two-pass approach, normalisation would have to be done online and the scale would shift as new data arrive.

4.3 Online Transform (`transform_daily`)

During the live simulation, each new day's signal is processed as follows:

1. Extract raw features $\mathbf{f}_i$ from the vibration signal.
2. Append to rolling buffer; compute 5-day smoothed features $\tilde{\mathbf{f}}_i$.
3. Select the monotonicity-approved features, normalise with $\boldsymbol{\mu}_{\text{train}}$ and $\boldsymbol{\sigma}_{\text{train}}$.
4. Project onto PC1: $s_i = \mathbf{w}_1^\top \mathbf{z}_i$.
5. Compute scaled HI: $\text{HI}_i^{\text{scaled}} = 1 - \delta_i / \delta_{\text{max}}$, clamped to $[0, 1]$.

6. Apply EMA: $\widehat{\text{HI}}_i = \alpha \cdot \text{HI}_i^{\text{scaled}} + (1 - \alpha) \widehat{\text{HI}}_{i-1}$, with $\alpha = 0.2$.

The EMA parameter $\alpha = 0.2$ was chosen to give 80% weight to the running history and 20% to the latest observation, which is appropriate for a once-per-day measurement cadence where genuine degradation is gradual relative to per-day noise fluctuations.

4.4 Feature Selection via Monotonicity

Only features that increase (or decrease) consistently over the healthy training window are informative for PCA-based health monitoring. The monotonicity score [1] quantifies this:

$$M_j = \frac{|\#\{\Delta \tilde{f}_{j,i} > 0\} - \#\{\Delta \tilde{f}_{j,i} < 0\}|}{N - 1} \quad (9)$$

$M_j = 1$ means perfectly monotonic; $M_j = 0$ means equal numbers of rises and falls. The threshold of 0.30 was selected to admit only features that are predominantly directional. As shown in Figure 3, RMS scores only 0.02 (essentially random walk during the healthy phase) and is excluded, while Kurtosis and Crest Factor both score 0.31 and are admitted.

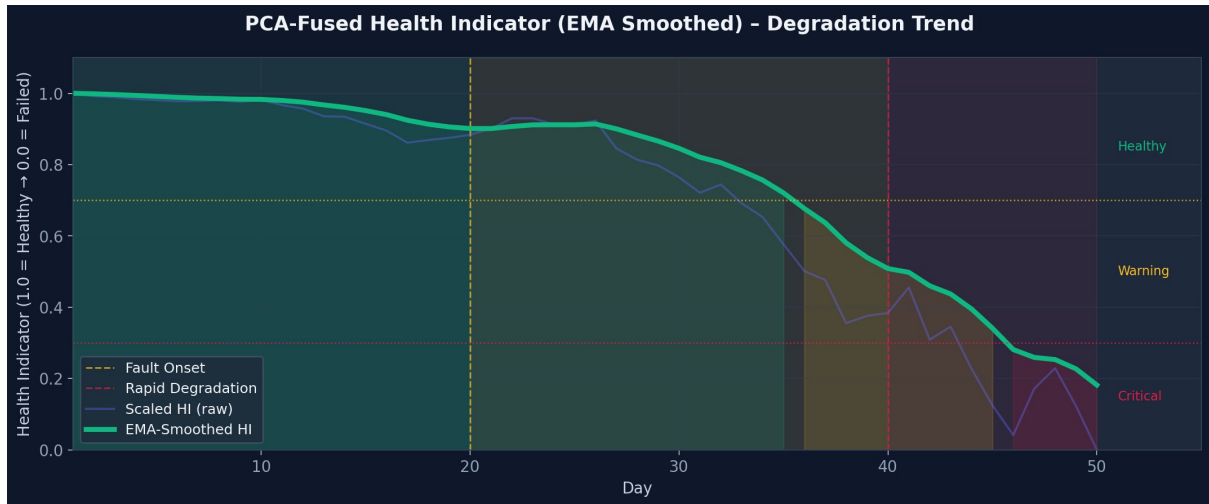


Figure 2. PCA-fused EMA-smoothed Health Indicator over 50 days. Colour-coded fill regions indicate health zones. The faint purple line shows the unsmoothed scaled HI for comparison.

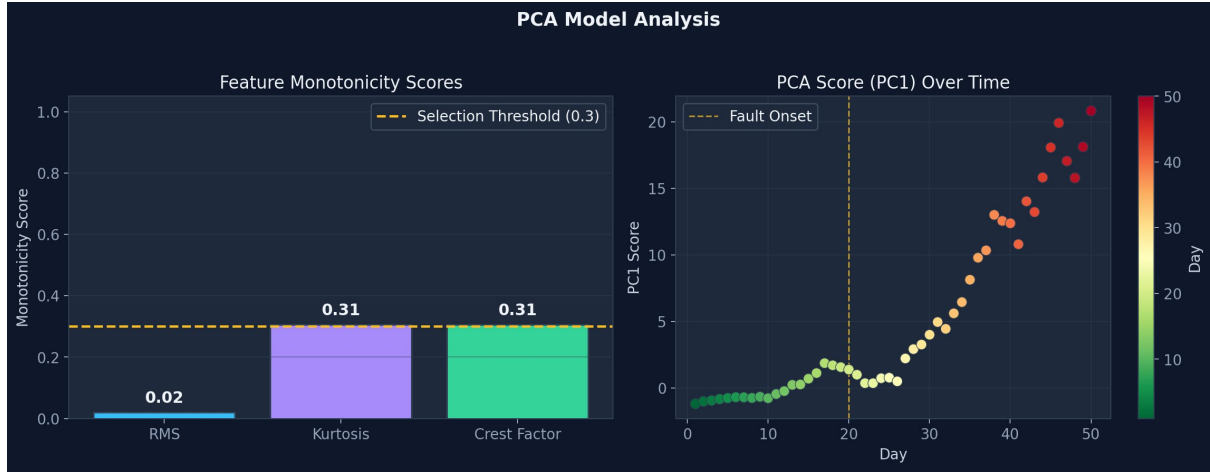


Figure 3. Left: Monotonicity scores for each feature; only Kurtosis and Crest Factor exceed the 0.30 threshold and are used in PCA. Right: PC1 score trajectory over 50 days, colour-coded by day number.

5 Frequency Spectrum Analysis

5.1 Bearing Fault Frequencies

When a rolling-element bearing has a localised defect on one of its surfaces, the rolling elements repeatedly impact the defect as the shaft rotates. Each impact produces a short impulsive vibration burst, and because the geometry of the bearing dictates how often each element passes over the fault, these impacts occur at predictable, calculable frequencies known as **bearing defect frequencies** (also called fault characteristic frequencies).

For a bearing with N_b rolling elements, pitch diameter D_p , ball diameter D_b , and a shaft rotating at frequency f_r (Hz), the key defect frequencies are:

$$\text{BPFI} = \frac{N_b}{2} f_r \left(1 + \frac{D_b}{D_p} \cos \phi \right), \quad \text{BPFO} = \frac{N_b}{2} f_r \left(1 - \frac{D_b}{D_p} \cos \phi \right) \quad (10)$$

where ϕ is the contact angle and BPFI / BPFO stand for *Ball Pass Frequency Inner race* and *Ball Pass Frequency Outer race* respectively. A fault on the inner race produces impulses at BPFI; a fault on the outer race at BPFO. The rolling-element spin frequency BSF (*Ball Spin Frequency*) governs defects on the ball surface itself.

For the 624ZZ-geometry bearing in this dataset, these evaluate to approximately: BPFI ≈ 162 Hz, BPFO ≈ 107 Hz, and BSF ≈ 68 Hz. Because the dataset has a confirmed **inner-race fault**, we expect energy to grow at integer multiples (*harmonics*) of BPFI as the fault progresses. The FFT spectra in Figure 4 overlay dashed vertical lines at the first four harmonics of each fault frequency purely as diagnostic reference markers – they are not used in the HI computation itself.

5.2 Spectral Evolution Across 50 Days

Figure 4 presents the FFT spectra at four key lifecycle stages.

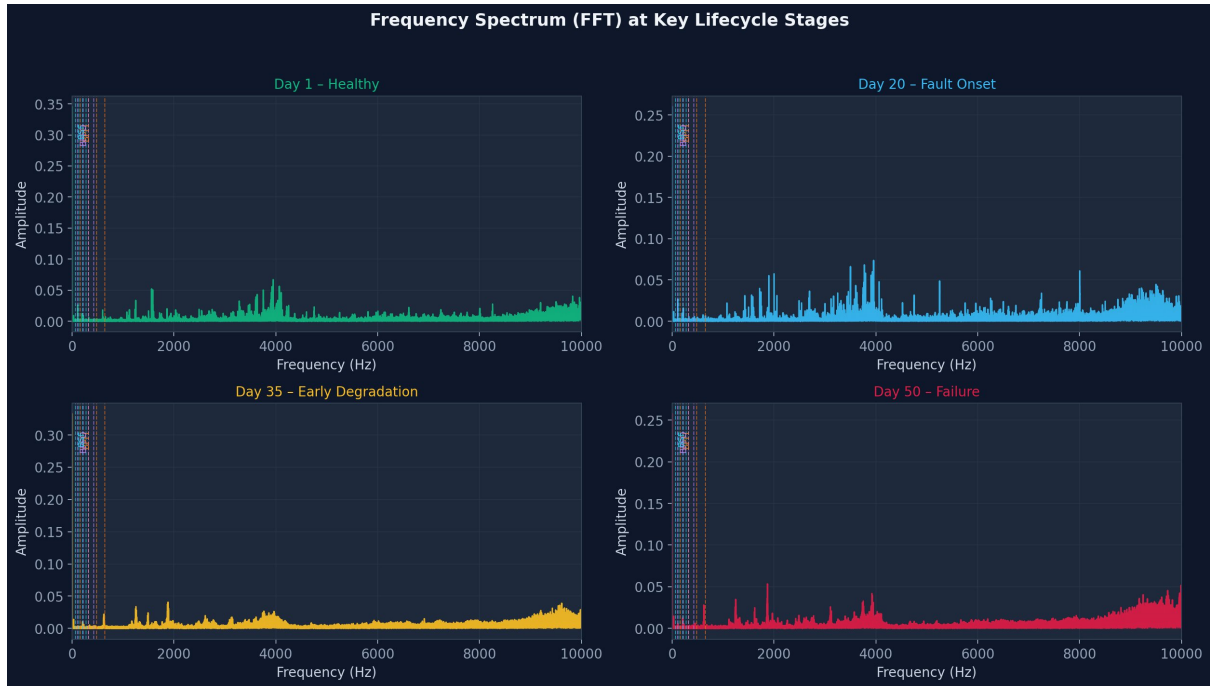


Figure 4. FFT amplitude spectra at days 1, 20, 35, and 50. Dashed vertical lines mark harmonics of BPF1 (orange), BPFO (purple), and BSF (cyan) as diagnostic reference. All spectra are truncated to 10 kHz.

In the healthy state (Day 1) the spectrum is dominated by low-amplitude broadband noise with no prominent tonal content. By Day 20, discrete peaks begin to emerge near the BPF1 fundamental and its second harmonic, consistent with the onset of inner-race surface fatigue. Days 35 and 50 show progressive growth of these harmonics accompanied by broadband noise-floor elevation across the 2–8 kHz range, which is characteristic of accelerating surface wear and lubricant deterioration.

Figure 5 provides the corresponding time-domain view. The healthy Day 1 signal is stationary Gaussian noise ($\text{RMS} = 2.297 \text{ g}$, $\text{Kurtosis} = 3.00$), while Day 50 exhibits clear impulsive bursts ($\text{Kurtosis} = 5.56$) with peak amplitudes exceeding $\pm 12 \text{ g}$.



Figure 5. Vibration waveforms for Day 1 (healthy) and Day 50 (failed). The impulsive ring-downs in the lower panel are characteristic of inner-race spalling.

6 System Architecture & Communication

6.1 Overall Architecture

The system follows a layered client–server Digital Twin architecture, illustrated in Figure 6. Data flows from raw `.mat` files through the signal-processing pipeline to the REST API, and from there to both the browser dashboard and the Fusion 360 3-D model.

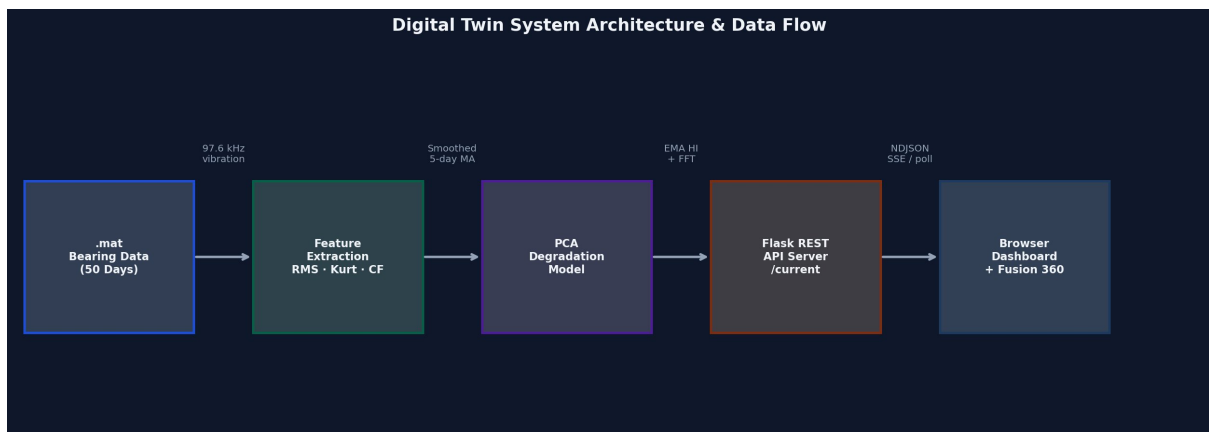


Figure 6. End-to-end system architecture and data flow.

6.2 Backend: Flask REST API on Raspberry Pi (server3.py)

The backend runs on a **Raspberry Pi** (serving as the edge compute node), bound to `0.0.0.0:5000` so that any device on the local network can reach it. It is implemented in

Python using the **Flask** web framework with **Flask-CORS** enabled, which allows the browser dashboard and the Fusion 360 script – potentially running on a different machine – to make cross-origin HTTP requests without being blocked by the browser’s same-origin policy.

6.2.1 Threading model and shared state

Flask is launched in `threaded=True` mode so that HTTP requests are handled concurrently. The simulation itself runs in a **separate daemon thread** (`stream_worker`). Both the HTTP worker threads and the simulation thread access a single shared Python dictionary `_current_state`. To avoid race conditions (e.g. the API thread reading a partially-written dictionary while the simulation thread is mid-update), every access to `_current_state` is wrapped in a `threading.Lock()`:

```

1 state_lock = threading.Lock()
2 _current_state = {}
3
4 # In the simulation thread:
5 with state_lock:
6     _current_state = result          # atomic replacement
7
8 # In the /current route:
9 with state_lock:
10     return jsonify(_current_state)  # consistent snapshot

```

Listing 1. Thread-safe state update (server3.py)

6.2.2 Simulation timing and data flow

The simulation thread reads each `.mat` file in day order, computes the full feature + PCA + FFT pipeline, writes the result to shared state, then sleeps for `STREAM_DELAY_S = 1.5s` before moving to the next day. This 1.5s inter-day pause was chosen to be human-perceptible on the dashboard while keeping the 50-day replay short enough for a live demonstration.

6.2.3 API endpoints

Table 2. Flask REST API endpoints

Method	Endpoint	Purpose
GET	/	Serves the dashboard HTML file
POST	/start-simulation	Spawns the simulation daemon thread
POST	/stop-simulation	Sets the stop flag; thread exits cleanly
GET	/current	Returns latest JSON state (HI, features, FFT)

The `/current` response is a flat JSON object, for example:

```

1 {
2   "day_index": 35,
3   "day_total": 50,
4   "health_indicator_pca": 0.6823,
5   "rms": 2.341,
6   "kurtosis": 3.97,
7   "crest_factor": 6.12,
8   "fft_freqs": [0.0, 20.0, 40.0, ...],    // ~500 points
9   "fft_amps":  [0.001, 0.002, ...]
10 }
```

Listing 2. Example `/current` JSON response

The FFT arrays are downsampled to ≈ 500 points (from $\sim 50,000$ raw FFT bins) before serialisation, keeping each JSON response below ~ 15 kB and ensuring comfortable transmission over a typical local Wi-Fi link between the RPi and client devices.

6.3 Frontend: Browser Dashboard (`index.html`)

The single-page dashboard is built with **Tailwind CSS** (CDN) and **Plotly.js**. It runs entirely in the browser and communicates with the RPi server exclusively through the `/current` polling endpoint – no WebSocket or server-sent events are required.

A JavaScript `setInterval` fires every 500 ms, issues a `fetch` to `/current`, and on a successful response:

- Updates five KPI cards: day index, PCA Health Indicator, Kurtosis, RMS, and Crest Factor.
- Dynamically re-colours the HI card border and trend chart fill based on the current health zone (green > 0.70 , yellow 0.30 – 0.70 , red < 0.30).
- Appends the new HI value to the Plotly area-chart trace (degradation trend).
- Replaces the FFT trace with the latest downsampled spectrum.

Because the dashboard is a static HTML file served directly by Flask, the user only needs to open `http://<RaspberryPi-IP>:5000` in any browser on the same network – no installation is required on the client side.

6.4 Fusion 360 3-D Visualisation (`Color.py`)

`Color.py` is a Python script that runs *inside* the Autodesk Fusion 360 scripting environment (which embeds its own Python interpreter). Its role is to bridge the Flask API on

the RPi to the 3-D CAD model, providing an intuitive live engineering view of bearing health.

6.4.1 Component discovery

On startup the script traverses the Fusion component tree recursively using `find_component(root, '624ZZ')` to locate the bearing component by name. Once found, a custom appearance called `DIGITAL_TWIN_LIVE_COLOR` is created by copying a base “Plastic – Matte (White)” material from the material library. This appearance is applied to every B-Rep body in the component, ensuring the entire bearing geometry is colour-controlled.

6.4.2 Polling and cache busting

The script enters an event loop that polls the RPi’s `/current` endpoint every 500 ms. Each request appends a Unix timestamp as a query parameter (`?t=1745123456.78`) and sets a `Cache-Control: no-cache` header. This *cache busting* is necessary because Fusion 360’s embedded HTTP client can otherwise serve a cached response, causing the colour to freeze even though the simulation is still running.

6.4.3 Health-to-colour mapping

The HI value is mapped to an RGB triplet with no blue component:

$$R = \lfloor (1 - \text{HI}) \times 255 \rfloor, \quad G = \lfloor \text{HI} \times 255 \rfloor, \quad B = 0 \quad (11)$$

When the bearing is healthy ($\text{HI} = 1.0$) the colour is pure green ($R = 0, G = 255, B = 0$). As health degrades the colour transitions smoothly through olive, yellow, and orange, reaching pure red ($R = 255, G = 0, B = 0$) at complete failure ($\text{HI} = 0.0$). The colour property is set via the Fusion API:

```

1 r = int((1.0 - hi_val) * 255)
2 g = int(hi_val * 255)
3 b = 0
4 color_prop.value = adsk.core.Color.create(r, g, b, 255)
5 app.activeViewport.refresh()
```

Listing 3. Live colour update in Fusion 360 (Color.py)

6.4.4 Day-change guard and completion

To avoid redundant material API calls (which are expensive in Fusion), the script tracks `last_day` and only applies a colour update when `day_index` changes. When `day_index >= day_total` (i.e. Day 50 is reached), the loop breaks and a modal dialog informs the

user that the simulation is complete. Live debug messages are printed to the Fusion *Text Commands* palette throughout.

6.4.5 Network error handling

Both `urllib.error.URLError` (network unreachable, RPi not yet started) and general exceptions are caught and logged to the Text Commands palette rather than crashing the script. This means the Fusion script can be started *before* the Flask server is running; it simply waits and retries each 500 ms cycle until the connection succeeds.

7 Results and Discussion

7.1 Health Indicator Performance

The EMA-smoothed PCA HI successfully tracks the three-phase degradation lifecycle: **stable health** (days 1–20, $\widehat{HI} \approx 0.97\text{--}1.00$), **gradual decline** (days 20–40, crossing the Warning threshold near day 37), and **accelerated failure** (days 40–50, entering the Critical zone near day 47 and reaching 0.19 at day 50). The EMA filter with $\alpha = 0.2$ successfully suppresses day-to-day noise while preserving the macroscopic degradation trend — the unsmoothed HI shows oscillations of up to ± 0.1 that the EMA largely eliminates.

7.2 Feature Analysis

The monotonicity analysis (Figure 3) reveals that RMS is a poor indicator in the training window, exhibiting near-random fluctuations during the healthy phase ($M_{\text{RMS}} = 0.02$). Kurtosis and Crest Factor both achieve a monotonicity score of 0.31, crossing the selection threshold. This is physically intuitive: kurtosis and crest factor respond to the impulse-like ball-pass events even before the bulk vibration energy (RMS) rises significantly.

7.3 Spectral Observations

The comparative FFT spectra confirm the inner-race fault progression discussed in Section 5. Discrete tonal peaks at the BPFI fundamental and its second harmonic (see Eq. 10) become clearly distinguishable by Day 20, and their amplitude grows through Days 35 and 50. Broadband noise floor elevation in the 2–8 kHz range at Day 50 is consistent with severe surface wear and lubricant degradation.

7.4 Digital Twin Communication Latency

The system achieves end-to-end latency from file processing to browser update of approximately 500 ms (the polling interval), which is well within the requirements for

quasi-real-time condition monitoring of slow-speed mechanical systems. The Fusion 360 colour update follows the same polling cycle, ensuring visual synchronisation between the dashboard and the 3-D model.

8 Key Code Explanation

8.1 Feature Extraction (`extract_time_features`)

```

1 def extract_time_features(self, raw):
2     raw = raw - np.mean(raw)           # Remove DC offset
3     rms = np.sqrt(np.mean(raw**2))     # Equation (1)
4     kurtosis = stats.kurtosis(raw, fisher=False) # Equation (2)
5     crest_factor = np.max(np.abs(raw)) / (rms + 1e-12) # Equation (3)
6     return np.array([rms, kurtosis, crest_factor])

```

Listing 4. Time-domain feature extraction (`server3.py`)

8.2 PCA Health Indicator Transform (`transform_daily`)

```

1 def transform_daily(self, raw_signal):
2     daily_features = self.extract_time_features(raw_signal)
3     self.raw_feature_history.append(daily_features)
4     history_window = self.raw_feature_history[-5:] # 5-day MA
5     smoothed_daily = np.mean(history_window, axis=0)
6
7     selected_daily = smoothed_daily[self.selected_features_idx]
8     normalized_daily = self.scaler.transform(selected_daily.reshape
9         (1, -1))
10
11     pca_val = self.pca.transform(normalized_daily)[0][0]
12
13     raw_hi = abs(pca_val - self.baseline_hi)
14     scaled_hi = 1.0 - (raw_hi / self.max_hi)
15     scaled_hi = max(0.0, min(1.0, scaled_hi)) # Clamp
16
17     alpha = 0.2 # EMA
18     self.ema_hi = alpha*scaled_hi + (1-alpha)*self.ema_hi
19     return self.ema_hi, daily_features

```

Listing 5. PCA transform and EMA filter (`server3.py`)

8.3 Fusion 360 Colour Mapping (`Color.py`)

```

1 r = int((1.0 - hi_val) * 255) # Red increases with degradation
2 g = int(hi_val * 255)         # Green decreases with degradation
3 b = 0
4 color_prop.value = adsk.core.Color.create(r, g, b, 255)

```

```
5 app.activeViewport.refresh()
```

Listing 6. RGB colour mapping from Health Indicator (Color.py)

9 Conclusion

This project demonstrates a fully operational Digital Twin for wind turbine bearing health monitoring. The system successfully integrates signal processing, machine learning (PCA), a REST API backend, an interactive browser dashboard, and a live-coloured 3-D Fusion 360 model into a coherent end-to-end pipeline.

The PCA-fused Health Indicator, guided by monotonicity-based feature selection and EMA smoothing, provides a robust and interpretable measure of bearing degradation that transitions through health zones in physical alignment with the known fault progression in the dataset. The system architecture is lightweight enough to run on a Raspberry Pi while remaining responsive at sub-second update rates.

Future work could incorporate exponential degradation models for quantitative Remaining Useful Life (RUL) prediction, adaptive retraining as new run-to-failure data becomes available, and multi-bearing parallelism for fleet-level monitoring.

References

- [1] MathWorks, *Wind Turbine High-Speed Bearing Prognosis*, MATLAB Predictive Maintenance Toolbox Documentation, 2024. <https://in.mathworks.com/help/predmaint/ug/wind-turbine-high-speed-bearing-prognosis.html>
- [2] MathWorks, *Principal Component Analysis (PCA)*, MATLAB Statistics and Machine Learning Toolbox Documentation, 2024. <https://in.mathworks.com/help/stats/principal-component-analysis-pca.html>
- [3] MathWorks, *Kurtosis – MATLAB Documentation*, MATLAB Statistics and Machine Learning Toolbox, 2024. <https://in.mathworks.com/help/stats/kurtosis.html>
- [4] ScienceDirect Topics, *Crest Factor*, Elsevier Engineering Reference, 2024. <https://www.sciencedirect.com/topics/engineering/crest-factor>